

Test Plan: RetryPlaywright Ecommerce Automation Project

Document version: 1.0
Date: July 6, 2026
Project: RetryPlaywright
Application under test: Rahul Shetty Academy Ecommerce Client
Primary URL: <https://rahulshettyacademy.com/client>

1. Objective

This test plan defines the test approach, scope, coverage, environments, risks, and execution strategy for the RetryPlaywright automation project. The project validates ecommerce user journeys using Playwright, TypeScript Page Object Model, JavaScript tests, API utilities, screenshots, network interception tests, and Cucumber BDD assets.

2. Test Scope

In Scope

- Login authentication for registered users.
- Negative login validation for unregistered users.
- Product search and add-to-cart flow.
- Cart product validation.
- Checkout and payment details entry.
- Order placement confirmation.
- Order history validation.
- API-assisted order placement and token-based authentication.
- Network interception and request/response validation.
- Screenshot and visual comparison examples.
- Cross-browser execution through Playwright projects: Chromium, Firefox, and WebKit.

Out of Scope

- Real payment processing.
- Production account management.
- Performance, load, and stress testing.
- Security penetration testing beyond existing API/network examples.
- Mobile device certification beyond configured Playwright viewport/device coverage.
- Backend data seeding outside the available API helper utilities.

3. Test Items

Area	Files/Assets	Purpose
TypeScript POM tests	tests/ClientAppPO.spec.ts , tests/LoginUnregisteredUser.spec.ts	End-to-end UI flows using TS page objects
Page objects	pageobjects_ts/*.ts	Encapsulate selectors and UI actions
Fixtures	utils_ts/test-base.ts	Reusable test data fixture
Test data	utils_ts/ClientAppData.json , utils_ts/ClientAppData2.json	Credentials and product data
API utilities	utils_ts/APIutils.js , utils/APIutils.js	Login/token and order API helpers

BDD	<code>features/Ecommerce.feature,</code> <code>features/step_definitions/steps.js</code>	Cucumber scenario coverage
Config	<code>playwright.config.js</code> , <code>playwright.config1.js</code>	Browser projects, retries, trace, screenshot and video policy

4. Test Strategy

The project uses Playwright for browser automation with a Page Object Model for maintainable UI tests. Tests should prioritize user-visible behavior and Playwright auto-waiting through locators and `expect` assertions. Selectors should be owned by page objects and reused by tests to avoid duplication.

1. Smoke tests: login, product add-to-cart, checkout confirmation.
2. Regression tests: full ecommerce order journey across configured browsers.
3. Negative tests: invalid login, unregistered login, empty required fields, invalid checkout values.
4. API tests: token generation, order creation, and response validation.
5. Network tests: request interception, blocked calls, and response mocking.

5. Entry Criteria

- Node dependencies are installed with `npm install`.
- Playwright browsers are installed.
- The ecommerce test site is reachable.
- Valid test credentials are available in fixture or test data files.
- Test data product names exist in the application catalog.
- No focused tests such as `test.only` are committed.

6. Exit Criteria

- Smoke suite passes in Chromium.
- Regression suite passes in all required browser projects or defects are documented.
- Critical and high severity failures are triaged.
- HTML report, traces, screenshots, or videos are available for failed tests.
- New tests use page objects, fixtures, and Playwright assertions.

7. Environment

Item	Value
Test runner	Playwright Test
Language	TypeScript and JavaScript
BDD framework	Cucumber.js
Browsers	Chromium, Firefox, WebKit
Reports	Playwright HTML report, Allure artifacts present in project
Default test directory	<code>./tests</code>
Default timeout	50 seconds
Assertion timeout	5 seconds

8. Test Data

Primary valid user data currently appears in `utils_ts/test-base.ts` and `utils_ts/ClientAppData.json`: username `martin.ampah@ashesi.edu.gh`, password `French123`, product `ZARA COAT 3`. Sensitive credentials should eventually be moved to environment variables or a secured secrets mechanism.

9. Test Coverage Matrix

Feature	Positive Coverage	Negative Coverage	Automation Status
Login	Registered user can login	Unregistered user cannot login	Partially automated
Product catalog	Product can be selected by name	Product unavailable case not covered	Automated for selected product
Cart	Product visible before checkout	Empty cart not covered	Automated
Checkout	Card, CVV, expiry, country, place order	Invalid payment fields not covered	Partially automated
Order confirmation	Thank-you message and order ID captured	Failed order not covered	Automated
Order history	Order ID lookup	Missing order handling not covered	Partially automated
API login/order	Token and order API flows	Invalid token/error response not fully covered	Partially automated
Network	Intercept, abort, response manipulation examples	Broader security cases not covered	Example coverage

10. Login Test Scenarios

- Successful login with registered email and valid password.
- Login attempt with unregistered email and valid-format password.
- Login attempt with registered email and incorrect password.
- Login attempt with invalid email format.
- Login attempt with blank email.
- Login attempt with blank password.
- Login attempt with both fields blank.
- Login error message is exposed as an accessible alert.
- Forgot password link navigates to password recovery.
- Register link navigates to registration.

11. Regression Test Cases

ID	Test Case	Expected Result	Priority
RP-001	Login with valid user	User lands on product dashboard	High
RP-002	Login with unregistered user	Error alert says <code>Incorrect email or password.</code> and user remains on login page	High
RP-003	Add existing product to cart	Product is added and visible in cart	High
RP-004	Checkout with valid details	Order is placed successfully	High
RP-005	Verify order in history	Order ID appears in order history/details	High
RP-006	API login creates token	API returns successful token response	Medium
RP-007	API order creation	Order is created through API helper	Medium
RP-008	Abort blocked network call	Blocked route does not load unwanted resource	Medium

RP-009	Screenshot comparison	Screenshot matches baseline	Low
--------	-----------------------	-----------------------------	-----

12. Execution Commands

Command	Purpose
<code>npm run regression</code>	Run full Playwright suite
<code>npm run webTests</code>	Run tests tagged <code>@Web</code>
<code>npm run APITests</code>	Run tests tagged <code>@API</code>
<code>npm run ChromiumConfig</code>	Run configured Chromium project from <code>playwright.config1.js</code>
<code>npx playwright test tests/LoginUnregisteredUser.spec.ts -project=chromium</code>	Run login negative test

13. Defect Management

Each defect should include title, severity, environment, browser, reproduction steps, actual result, expected result, test data, screenshots/video/trace/network evidence, and the related test or spec file.

14. Risks and Mitigations

Risk	Impact	Mitigation
External training site availability	Tests may fail due to outage or latency	Use API health checks and retry only in CI where appropriate
Hardcoded credentials	Data leakage and maintainability issues	Move credentials to environment variables
Hardcoded URL in page object	Harder environment switching	Use Playwright <code>baseURL</code>
CSS-heavy selectors	Fragile tests after UI changes	Prefer roles, labels, placeholders, and centralized page object selectors
Manual waits/networkidle usage	Flaky timing	Prefer locator assertions and Playwright auto-waiting
Mixed JS and TS implementations	Duplicate maintenance	Consolidate primary framework direction

15. Reporting

Playwright HTML report should be used for local debugging. For failed tests, retain traces, screenshots, and videos where configured. Allure artifacts are present in the project and can be used if the reporting workflow is maintained.

16. Recommendations

- Rename `ValidLogin` and `InvalidLogin` to a single `login` action and assert outcome in the test.
- Move login-negative test data to a login-specific fixture.
- Strengthen login error assertion to exact text.
- Replace duplicated inline selectors in tests with page object methods.
- Remove `page.pause()` from committed regression tests.
- Avoid `waitForLoadState('networkidle')` unless no reliable UI assertion exists.
- Add dedicated BDD coverage for the Login feature.